

Google Analytics Connector for SailPoint IdentityIQ: Technical Reference

Target system: Google Analytics 4 (GA4) Admin API, `v1alpha` access bindings
IdentityIQ version: 8.4 (local sandbox: Tomcat 9, MySQL 8, JDK 17)
Connector: Web Services (`sailpoint.connector.webservices.WebServicesConnector`)
Scope: account aggregation, correlation, and provisioning (create, add role, remove role, delete)
Status: built and tested end to end against a live GA account on 2026-10-06

1. Summary

This integration lets IdentityIQ govern who has access to a Google Analytics account. Every user who has been granted access to the GA account appears in IIQ as an account on the **Google Analytics** application, and each GA role (Viewer, Analyst, Editor, Administrator, and so on) appears as a requestable entitlement.

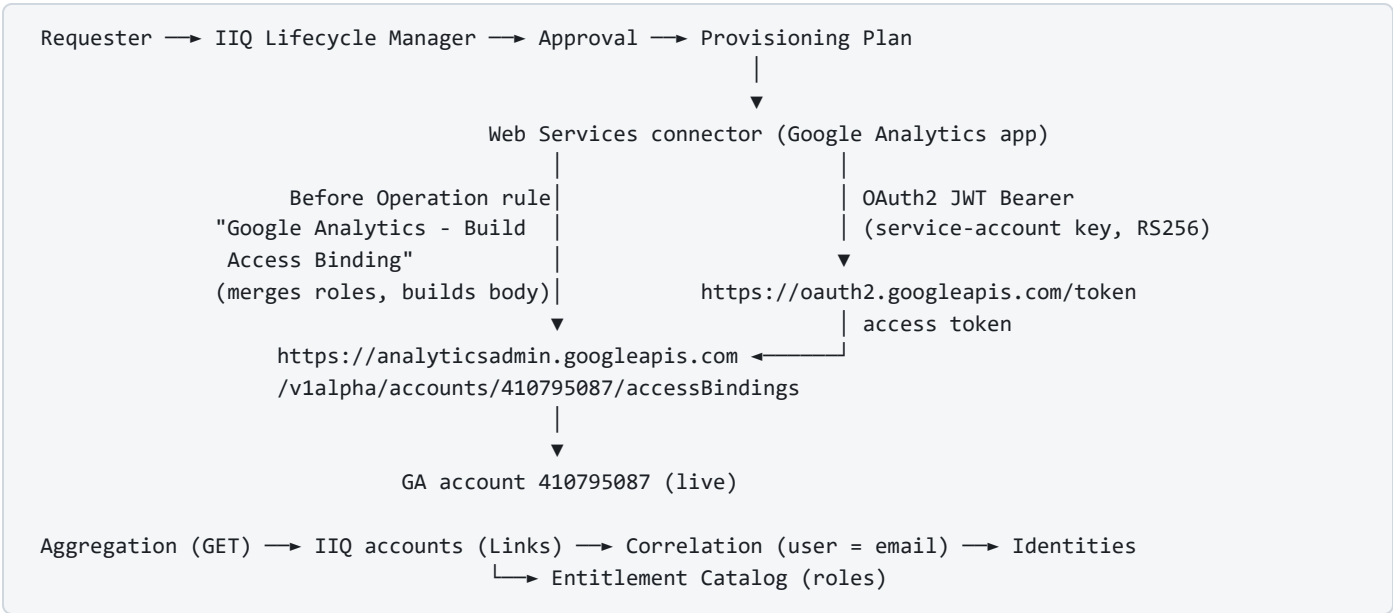
When someone requests a GA role through IIQ's Lifecycle Manager (LCM), IIQ calls the GA Admin API directly to grant it. Removing roles works the same way, and removing a user's last role deletes their GA access entirely.

There is no out-of-the-box SailPoint connector for Google Analytics, so the integration is built on the generic Web Services connector plus one BeanShell rule. The rule exists because GA's update API replaces a user's whole role list, while IIQ only sends the change (see [section 7](#)).

What was proven

Operation	HTTP call	Tested result
Test Connection	<code>GET .../accessBindings?pageSize=1</code>	Pass
Account Aggregation	<code>GET .../accessBindings?pageSize=500</code>	Pass. Accounts and roles land in IIQ, and roles are promoted to the Entitlement Catalog.
Create Account	<code>POST .../accessBindings</code>	Pass. The user appears in GA with the requested role.
Add Entitlement	<code>PATCH /v1alpha/{binding}</code>	Pass. The rule sent <code>[viewer, analyst]</code> , keeping the existing role.
Remove Entitlement	<code>PATCH /v1alpha/{binding}</code>	Pass. The rule sent <code>[analyst]</code> after Viewer was removed.
Remove last role	<code>DELETE /v1alpha/{binding}</code> (switched from PATCH by the rule)	Pass. The binding was removed from GA.
Delete Account	<code>DELETE /v1alpha/{binding}</code>	Configured, not yet run on its own. It is a plain DELETE to the same URL as the tested path above.

2. Architecture



Data model

GA4 grants account access through **access bindings**. One binding = one Google user + a list of roles on one GA account. A binding looks like this:

```
{
  "name": "accounts/410795087/accessBindings/AYVX2ezT4gkz-...",
  "user": "someone@gmail.com",
  "roles": ["predefinedRoles/viewer", "predefinedRoles/analyst"]
}
```

That maps cleanly onto IIQ:

GA concept	IIQ concept	Schema attribute
Access binding resource name (generated by Google)	Account native identity	<code>bindingName</code> (identity attribute)
<code>user</code> (Google account email)	Account display name, correlation key	<code>user</code> (display attribute)
<code>roles</code>	Multi-valued, managed entitlement	<code>roles</code>

`bindingName` is used as the native identity rather than the email because it is the value every update and delete URL needs. The email is only accepted on create.

GA predefined roles

Value in IIQ	GA UI name
<code>predefinedRoles/viewer</code>	Viewer

Value in IIQ	GA UI name
<code>predefinedRoles/analyst</code>	Analyst
<code>predefinedRoles/editor</code>	Editor
<code>predefinedRoles/admin</code>	Administrator
<code>predefinedRoles/no-cost-data / no-revenue-data</code>	Data restrictions (optional)

Only roles that some user already holds are promoted to the catalog by aggregation. Others can be added to the Entitlement Catalog by hand (application *Google Analytics*, attribute `roles`).

3. Google-side prerequisites

1. **Google Cloud project** (sandbox used `api-test-345702`) with the **Google Analytics Admin API** enabled.
2. **Service account** in that project (sandbox used `iiq-ga-reader@api-test-345702.iam.gserviceaccount.com`). The name says "reader" because it started read-only. Its permissions come from the GA role, not the name.
3. **Service account key** downloaded and converted to an **encrypted PKCS#8 PEM** with a password. That is the format the IIQ JWT form accepts (Private Key + Private Key Password). The key is never stored in the repo.
4. **GA account access**: in GA, go to *Admin* → *Account access management* and add the service account email with the **Administrator** role. Viewer is enough for aggregation only. Writing other users' access needs Administrator.
5. **OAuth scope**: `https://www.googleapis.com/auth/analytics.manage.users` . This scope covers both reading and writing. The read-only scope `analytics.manage.users.readonly` is only enough for aggregation.

A request is allowed only when **both** the scope (what the token asks for) and the GA role (what the service account actually has) allow it.

4. Application configuration

File: `config/Application/GoogleAnalytics.xml`

4.1 Features

```
featuresString="NO_RANDOM_ACCESS, PROVISIONING, SYNC_PROVISIONING"
```

Flag	Why
<code>NO_RANDOM_ACCESS</code>	There is no "get one account" operation. IIQ only reads accounts through full aggregation.
<code>PROVISIONING</code>	Tells IIQ to send provisioning plans to the connector instead of creating manual work items.

Flag	Why
SYNC_PROVISIONING	GA applies changes immediately, so results are final ("committed") rather than queued.

ENABLE , UNLOCK and PASSWORD are deliberately absent. A GA binding can't be disabled and has no password.

A feature flag is only a promise. Every flagged capability needs a matching connector operation, otherwise provisioning fails instead of falling back to a manual work item.

4.2 Authentication (OAuth2 JWT Bearer)

Attribute	Value
authenticationMethod	OAuth2Login
grant_type	JWT_BEARER
token_url	https://oauth2.googleapis.com/token
oAuthJwtHeader	alg = RS256
oAuthJwtPayload.aud	https://oauth2.googleapis.com/token
oAuthJwtPayload.iss	service account email
oAuthJwtPayload.exp	15f (15 minutes)
oAuthJwtPayload.scope	https://www.googleapis.com/auth/analytics.manage.users
private_key , private_key_password	entered in the UI, encrypted by IIQ, not in the repo

The connector builds and signs the JWT, exchanges it at Google's token endpoint, and caches the access token (oauth_token_info , encrypted).

Where is the scope in the UI? The IIQ 8.4 Web Services form for the JWT grant type shows only Token URL, Private Key and Private Key Password. The JWT payload (including scope and iss) is stored in the oAuthJwtPayload map and is edited on the **Debug page** (/identityiq/debug → Application → *Google Analytics*). See [11.3](#) for the trap this creates.

4.3 Connection

Attribute	Value
genericWebServiceBaseUrl	https://analyticsadmin.googleapis.com
enableHasMore	true
fixedPlanMultivaluedAttribute	true

4.4 Schema (account)

Attribute	Type	Flags	Notes
bindingName	string	identity attribute	accounts/410795087/accessBindings/<id>

Attribute	Type	Flags	Notes
user	string	display attribute	Google account email
roles	string	entitlement, managed, multi-valued	GA predefined roles

`nativeObjectType="user" , objectType="account" .`

5. Connector operations

All operations live in the `connectionParameters` list. Settings that apply to every operation: body format `raw` , success code `2**` .

#	Operation	Method	Context URL	Before rule	Response mapping
1	Test Connection	GET	<code>/v1alpha/accounts/410795087/accessBindings?pageSize=1</code>	none	<code>root \$.accessBindings</code>
2	Account Aggregation	GET	<code>/v1alpha/accounts/410795087/accessBindings?pageSize=500</code>	none	<code>root \$.accessBindings ; bindingName←name , user←user , roles←roles</code>
3	Create Account	POST	<code>/v1alpha/accounts/410795087/accessBindings</code>	Build Access Binding	<code>root \$; bindingName←name , user←user , roles←roles</code>
4	Add Entitlement	PATCH	<code>/v1alpha/\$plan.nativeIdentity\$</code>	Build Access Binding	none
5	Remove Entitlement	PATCH	<code>/v1alpha/\$plan.nativeIdentity\$</code>	Build Access Binding	none
6	Delete Account	DELETE	<code>/v1alpha/\$plan.nativeIdentity\$</code>	none	none

Operations 3 to 5 send the header `Content-Type: application/json` . Their body is left empty in the configuration because the rule writes it at run time.

Notes per operation

- **Aggregation paging.** The API's maximum page size is 500 and the sandbox account has far fewer bindings, so there is a single request. If an account grows past 500, add a paging step on `nextPageToken` .
- **Create response mapping is essential.** Google generates the binding ID. The response mapping is the only way IIQ learns the new account's native identity. The schema attribute name must match **exactly** (`bindingName` , not `BindingName`). See [11.6](#).
- `$plan.nativeIdentity$` is replaced with the account's `bindingName` . The connector URL-encodes it (`accounts%2F410795087%2FaccessBindings%2F...`), and Google accepts the encoded form.
- **Delete Account** is used by explicit deletes (Manage Accounts, leaver events). Removing the *last* role does not use this operation. It goes through Remove Entitlement, and the rule switches the method to DELETE.

6. Provisioning policy (Create Account form)

```
<Form name="Google Analytics Create Account" objectType="account" type="Create">
  <Section name="Section 1">
    <Field displayName="Google Email" name="user" required="true" type="string">
      <Script><Source>return identity.getEmail();</Source></Script>
    </Field>
  </Section>
</Form>
```

- The field `name` must equal the schema attribute (`user`), so the value lands in the plan as an `AttributeRequest` the rule can read.
- Type is **string**. The *Identity* field type would hold an identity reference, not an email.
- No owner and not dynamic, so it is calculated silently and never shown to a person.
- The IIQ 8.4 field editor has no "identity attribute" value source, so a one-line **Script** is used.

Hard requirement: the identity's email must be a real Google account. If it isn't, GA returns `404 Not Found` on create (see [11.5](#)).

7. The Build Access Binding rule

File: `config/Rule/Rule-GoogleAnalytics-BuildAccessBinding.xml`

Type: `WebServiceBeforeOperationRule` . Attached as the Before Rule on Create Account, Add Entitlement and Remove Entitlement.

7.1 The problem it solves

GA's `PATCH accessBindings/{id}` **replaces** the binding's whole `roles` array. An IIQ provisioning plan only carries the *change* ("add analyst"). Sending just the change would wipe every role the user already had. The Web Services connector can't merge lists on its own, so a rule has to.

7.2 Algorithm

1. Find the `AccountRequest` for the *Google Analytics* application in the plan. If there is none, return the endpoint unchanged.
2. **Starting roles.** For a Create, start empty. Otherwise, look up the `Link` (IIQ's stored copy of the account) by application name + native identity and read its `roles` .
3. **Apply the plan.** For each `AttributeRequest` : - `user` : remember it (used only on create). - `roles` with `Remove` : remove those values. - `roles` with `Set` : clear, then add. - `roles` with `Add` (or no op): add any values not already present.
4. **Build the request:** - Roles remain: write `{"user": ..., "roles": [...]}` into `body.jsonBody` , including `user` only on create. - No roles remain on an update: switch the method to **DELETE** and clear the body. GA rejects a binding with no roles, so removing the last role means removing access. - No roles on a

create: throw an error, because there is nothing to grant. - A create with no user : throw an error pointing at the provisioning policy.

5. Log the final method, URL and role list at debug level, then return the endpoint.

7.3 Properties worth knowing

- **Same input, same request.** The rule always sends the complete final list. If the connector calls the endpoint once per entitlement value in a multi-role request, each call sends the same body, so nothing breaks.
- **Its view of current roles can be out of date.** "Current roles" means IIQ's last aggregated copy. If someone changes access in the GA UI directly, IIQ doesn't know until the next aggregation, and a PATCH in between can undo that change. To keep this risk small, aggregate on a schedule. A stricter design would `GET` the binding inside the rule before patching.
- **No credentials handled.** Authentication is done by the connector's built-in OAuth2 JWT support. The rule only builds the body.

7.4 Proof from the debug log

```
10:35:58 POST    .../accounts/410795087/accessBindings          roles=[predefinedRoles/viewer]
10:41:36 PATCH   .../accessBindings%2FAYVX2ezT4gkz-...           roles=[predefinedRoles/viewer,
predefinedRoles/analyst]
10:42:19 PATCH   .../accessBindings%2FAYVX2ezT4gkz-...           roles=[predefinedRoles/analyst]
10:52:40 no roles left on accounts/410795087/accessBindings/AYVX2ezT4gkz-..., deleting the binding
10:52:40 DELETE  .../accessBindings%2FAYVX2ezT4gkz-...
```

8. Correlation

File: [config/CorrelationConfig/GoogleAnalyticsCorrelation.xml](#)

```
<CorrelationConfig name="GoogleAnalyticsCorrelation">
  <AttributeAssignments>
    <Filter operation="EQ" property="email" value="user"/>
  </AttributeAssignments>
</CorrelationConfig>
```

An account's `user` email is matched against the identity attribute `email`. Correlation fails (the account gets its own new "uncorrelated" identity) when:

- no identity has that email,
- **more than one** identity has that email (IIQ refuses to guess), or
- the account was first aggregated *before* a matching email existed. Later aggregations usually don't move an existing account to a different identity. Fix this once by hand under *Identities* → *Identity Correlation*.

Accounts created **through IIQ** don't depend on correlation at all. The Create response mapping attaches them to the requesting identity directly.

9. Tasks and keeping things in sync

Task	Purpose
Agg-GoogleAnalytics (file)	Account aggregation with <code>checkDeleted=true</code> (removes IIQ accounts whose binding is gone) and <code>promoteManagedAttributes=true</code> (adds roles to the catalog)
Refresh Identity Cube, with Refresh identity entitlements for all links	Updates the identity's entitlement list that LCM's <i>Remove Access</i> tab reads
Full Text Index Refresh	Makes new or newly requestable entitlements searchable in <i>Manage User Access</i>
Perform Identity Request Maintenance	Moves access requests from <i>Verifying</i> to <i>Completed</i> once aggregation confirms the change

IIQ keeps **two copies** of who has what:

1. The **Link** (application account): updated by aggregation and by provisioning results.
2. The **IdentityEntitlement** table (the identity's entitlements list): updated by identity refresh. LCM's *Remove Access* list reads this one.

They can disagree until a refresh runs. That was seen in testing: the account showed `analyst` while Remove Access still offered `viewer`. Recommended routine after any GA aggregation: **Agg-GoogleAnalytics** → **Refresh Identity Cube (with entitlements)** → **Full Text Index Refresh**, ideally as one sequential task.

10. Lifecycle Manager (request side)

- In the **Entitlement Catalog**, mark the `roles` values **Requestable**, with readable display names.
- Run **Full Text Index Refresh**. LCM search reads a full-text index, not the catalog directly.
- Make sure LCM allows entitlement requests for the requester population (*Global Settings* → *Lifecycle Manager*).
- For leavers, use **delete**, not **disable**. The app has no `ENABLE` feature, so a **disable** becomes a manual work item.
- Access requests go **Success / Finished** → **Verifying** → **Completed**. *Verifying* is normal. It ends after aggregation plus Perform Identity Request Maintenance.

11. Troubleshooting log (everything that went wrong, and why)

These are real issues hit while building the integration, in the order they happened.

11.1 Test Connection: 503 Service Unavailable

Cause: wrong private key password.

How it was found: an unauthenticated `curl` to the same URL returned a normal `401`, so the API was up. The

503 was IIQ's misleading way of reporting a token failure.

Lesson: check the key password before believing that Google is down.

11.2 Read-only scope blocked provisioning

The original scope was `analytics.manage.users.readonly`, which is fine for aggregation. Writes need `analytics.manage.users`. Test Connection only reads, so it **passes with either scope**, and only a write proves the scope is right.

11.3 Debug page edits lost after saving in the UI

The scope was changed on the Debug page while the application editor was open. Saving in the editor later wrote its older in-memory copy back over the change.

Rule: close the app editor before making Debug page edits, and reload the object afterwards to confirm.

11.4 New operations disappeared

Operations were added and saved inside the operation editor, but the **application** was never saved, so nothing reached the database. An exported XML showed only the original two operations.

Rule: an operation's *Save* only keeps it in your editing session. Click **Save** on the main Edit Application page, then reopen the app and confirm.

11.5 404 Not Found on create

The sandbox identities' emails were made-up addresses. GA can only grant access to **real Google accounts**, and answers "not found" for anything else.

Fix: set the test identity's email to a real Gmail. Note that an HR aggregation or refresh may restore the original email if HR is the source of that attribute.

11.6 "Native identity is neither present in the plan nor in the response"

Google **accepted** the create (the result was `committed`, and the message is a *warning*), but the Create Account response mapping said `BindingName` instead of `bindingName`. Schema attribute names are case-sensitive, so IIQ never learned the new account's ID.

Fix: correct the mapping. Don't resubmit the request, because the binding already exists and a second create would conflict. Aggregate instead.

11.7 Account didn't correlate after aggregation

The `user` email and identity email matched, but the account still came in uncorrelated. The likely cause is timing (see [section 8](#)). It was fixed by moving the account under *Identities* → *Identity Correlation*.

11.8 Request stuck on "Verifying"

Not stuck. IIQ waits for aggregation to confirm the change. Running Agg-GoogleAnalytics + Perform Identity Request Maintenance moved it to *Completed*.

11.9 "Filtered Items" on a remove request

The remove-Viewer item also appeared under *Filtered Items*. This turned out to be harmless display behavior. The debug log showed the correct PATCH ([analyst]) was sent.

11.10 Remove Access showed the wrong role

After Viewer was removed, the identity's Remove Access list still showed `viewer` while the account showed `analyst`. That was the two-copies issue from [section 9](#). Fixed by Refresh Identity Cube with *Refresh identity entitlements for all links*.

11.11 Request not visible in Manage User Access

The entitlement was requestable but not searchable until **Full Text Index Refresh** ran.

12. Debug logging recipe

The Web Services connector and the rule's `log.debug` output both go to the `sailpoint.connector.webservices` logger. To capture them in a dedicated file, append this to `WEB-INF/classes/log4j2.properties`. IIQ reloads the file every 20 seconds (`monitorInterval=20`), so no restart is needed.

```
appender.gaws.type=File
appender.gaws.name=gawsFile
appender.gaws.fileName=C:/IIQSandbox/tomcat/logs/ga-ws.log
appender.gaws.layout.type=PatternLayout
appender.gaws.layout.pattern=%d{ISO8601} %5p %t %c{4} - %m%n
logger.gaws.name=sailpoint.connector.webservices
logger.gaws.level=debug
logger.gaws.additivity=false
logger.gaws.appenderRef.gaws.ref=gawsFile
```

This logs the full application configuration and provisioning plans, which is what diagnosed 11.6. Encrypted values appear encrypted, but the log still describes the app in detail, so remove the logger when you're done.

Useful search:

```
grep -aE "Google Analytics: (POST|PATCH|DELETE|no roles)" ga-ws.log
```

13. Known limitations and future work

- **v1alpha API**. Google may change or replace it. Watch for a `v1beta` / `v1` access bindings release.
- **Out-of-date role view**. See [7.3](#). A GET before each PATCH would remove the risk.
- **Single GA account**. The account ID `410795087` is hard-coded in URLs. Supporting several accounts would mean one IIQ application per GA account, or a property-level design.
- **Account level only**. Property-level bindings (`properties/{id}/accessBindings`) are not managed.

- **No paging** beyond 500 bindings.
- **Emails must be Google accounts.** People without one can't be provisioned. A provisioning policy check or request-time validation would help.
- **Not yet done:** a standalone Delete Account test through Manage Accounts, a scheduled sequential sync task, certification campaigns for GA roles, and catalog display names.

14. File inventory

File	Purpose
config/Application/GoogleAnalytics.xml	Application: auth, six operations, schema, provisioning policy (secrets removed)
config/Rule/Rule-GoogleAnalytics-BuildAccessBinding.xml	Before Operation rule: role merge, body build, PATCH→DELETE switch
config/CorrelationConfig/GoogleAnalyticsCorrelation.xml	Correlation user → identity email
config/TaskDefinition/Agg-GoogleAnalytics.xml	Account aggregation task
docs/google-analytics/TECHNICAL.md / .pdf	This document
docs/google-analytics/STEP-BY-STEP.md / .pdf	Beginner rebuild guide

Import order: Rule → CorrelationConfig → Application → TaskDefinition. Then re-enter the private key and password in the UI and save the application.